• SOFTWARE ENGINEERING LAB

LAB 19 // CORE ARCHITECTURE

Database Connection Pooling & Limits

Connection Overhead Analysis, Hard Server Limits, and Leak Mitigation Under Concurrent Load in Go

Lukman / lukman-ss



Masalah: Koneksi Habis di Beban Tinggi

Koneksi database bertumbukan saat aplikasi menerima lonjakan konkurensi.

MOTIVASI

Aplikasi sering kali crash dengan pesan 'FATAL: sorry, too many clients already' pada PostgreSQL. Respons spontan tim biasanya memperbesar pool aplikasi atau menaikkan max_connections server tanpa menyadari efek sampingnya.

BATAS LAB

Lab ini membuktikan simulasi driver Go database/sql: membedakan perilaku pembukaan soket baru yang berbiaya mahal versus penggunaan kembali koneksi yang efisien.

Connections are scarce backend processes, not disposable application threads.

Mengapa Koneksi Berbiaya Sangat Mahal?

Inisialisasi soket menuntut TCP handshake, TLS, dan proses worker server.

ANATOMI OVERHEAD

Membuka koneksi baru menuntut three-way TCP handshake, negosiasi TLS/SSL, otentikasi kredensial, dan inisialisasi alokasi RAM per proses worker di database engine.

DAMPAK OPERASIONAL

Koneksi berumur pendek (< 60 detik) membakar CPU database semata untuk membuka/memutus koneksi. Azure mencatat koneksi idle sekalipun tetap memakan kuota RAM.

Every new connection pays an unavoidable network and memory initialization tax.

Mental Model: Meja Restoran & Dapur

Kapasitas throughput ditentukan oleh dapur, bukan banyaknya kursi meja.

ANALOGI SISTEM

Koneksi database ibarat meja di restoran. Menambah meja melebihi kapasitas dapur tidak menambah makanan yang dimasak, melainkan hanya memperpanjang antrean dan waktu tunggu.

PRINSIP HARDWARE

Kapasitas ditentukan oleh batas core CPU dan I/O disk server database, bukan oleh jumlah goroutine atau thread pemanggil di lapisan aplikasi klien.

Throughput is bounded by database hardware, not by client thread counts.

Kurva Kejenuhan (Saturation Knee)

Melewati titik jenuh membuat throughput anjlok drastis akibat kontensi.

MEKANISME KONTENSI

PostgreSQL Wiki merinci 6 biang keladi degradasi: disk contention, tekanan RAM (work_mem x koneksi), lock contention, context switching CPU, cache bouncing, dan O(N²) scaling.

TEMUAN PENELITIAN

Pertanyaan arsitektural yang benar bukan 'sebesar apa pool yang aman?', melainkan 'sekecil apa pool yang masih sanggup melayani kebutuhan throughput aplikasi?'

Beyond the saturation point, adding connections degrades total system throughput.

Formula Kapasitas: Core & Spindle

Titik awal penentuan ukuran pool berbasis hardware menurut PostgreSQL & HikariCP.

FORMULA EMPIRIS

connections = ((core_count * 2) + effective_spindle_count). Server 4-core dengan 1 disk menghasilkan 9-10 koneksi optimal untuk memaksimalkan throughput CPU tanpa thrashing.

BATASAN REALISTIS

Formula ini divalidasi pada era HDD. Pada NVMe SSD modern, formula ini tetap menjadi acuan baseline awal yang wajib divalidasi lewat pengujian beban terukur.

Start sizing from hardware limits: ((cores * 2) + spindles), then load-test.

Multiplikasi Multi-Instance Pod

Ukuran pool harus dihitung pada skala deployment agregat, bukan per pod.

MATEMATIKA DEPLOYMENT

Total koneksi = Jumlah Replica Pod x Ukuran Pool per Pod. 4 pod microservice x 50 koneksi = 200 koneksi serentak ke single primary database.

RISIKO EXHAUSTION

Jika server database disetel max_connections = 100, lonjakan trafik otomatis menyebabkan separuh pod gagal terkoneksi dan memicu cascading service failure.

Always calculate total connection demand across the entire distributed cluster.

Arsitektur Lab: In-Memory Mock Driver

Mensimulasikan handshake latency dan kapasitas server secara deterministik.

KOMPONEN DRIVER

internal/pool/mockdb.go menerapkan database/sql/driver. Driver mengelola atomic activeConns, batas maxConnections, dan jeda inisialisasi connectDelay.

MANFAAT SIMULASI

Menghilangkan dependensi eksternal PostgreSQL, memungkinkan assertions deterministik pada kondisi overloaded, starvation, dan context cancellation di unit test.

Deterministic driver simulation enables reliable isolation of failure modes.

Implementasi: MockDriver & mockConn

Pelacakan atomic counters dan perlindungan penutupan soket ganda.

LOGIKA PENERIMAAN

MockDriver.Open mengecek kuota: jika activeConns >= maxConnections, kembalikan ErrServerOverloaded. Jika aman, naikkan activeConns dan totalCreated.

IDEMPOTENSI CLOSE

mockConn.Close dilindungi mutex dan flag closed bool. Penutupan ganda dijamin idempoten tanpa memicu kepanikan double-decrement counter (TestMockConnDoubleClose).

Safe resource tracking requires strictly idempotent connection release mechanisms.

Uji 1: Overhead Pooled vs Unpooled

Pembuktian empiris efisiensi penggunaan kembali soket terbuka di Go.

KODE & PENGATURAN

SetMaxIdleConns(0) memaksa pemutusan koneksi pasca-eksekusi (unpooled). SetMaxIdleConns(5) mempertahankan koneksi terbuka di pool untuk dipakai ulang.

HASIL VERIFIKASI

TestDirectConnectionOverhead membuktikan pooled puluhan kali lebih cepat. Pada demo CLI: unpooled (5 req) memakan 54.59ms, sedangkan pooled selesai dalam 6.33µs.

Connection pooling eliminates the repetitive handshake penalty across requests.

Uji 2: Server Exhaustion Rejection

Ketika alokasi pool klien melampaui batas keras kapasitas server.

SKENARIO UJI

Server disetel max 10 koneksi, namun klien mengizinkan pool membuka 20 koneksi (db.SetMaxOpenConns(20)). 20 goroutine mengeksekusi kueri serentak.

BUKTI ASSERTION

TestOversizedPoolExhaustsServerConnections menegaskan minimal 10 request ditolak dengan ErrServerOverloaded. Klien tidak bisa memaksakan koneksi yang ditolak server.

An oversized client pool inevitably crashes against hard database server limits.

Anatomi Kebocoran: Menahan Koneksi

Penyebab nomor satu starvasi koneksi pada sistem backend monolitik & mikroservis.

ANTI-PATTERN KODE

ProcessOrderUnsafeLeak meminjam koneksi database, menjalankan UPDATE, lalu memanggil externalCall() lambat (HTTP API 100ms) sebelum melepaskan koneksi.

DAMPAK STARVASI

Selama pemanggilan I/O jaringan berlangsung, soket database terkunci pasif. Koneksi tidak melayani kueri apa pun namun tidak dapat digunakan goroutine lain.

Never hold a database connection while waiting for network I/O or external APIs.

Uji 3: Starvasi Pool Akibat Leak

Satu kode bocor menghancurkan seluruh antrean request sehat di aplikasi.

SKENARIO ISOLASI

Pool disetel size = 1. Request pertama menjalankan ProcessOrderUnsafeLeak dengan jeda 100ms. Request kedua memanggil ProcessOrderSafe yang bersih dengan timeout 30ms.

TEMUAN FATAL

TestConnectionStarvationDueToLeak membuktikan request kedua gagal dengan context.DeadlineExceeded. Request sehat ikut mati akibat koneksi tertahan di request lain.

A single leaky handler creates cascading starvation across independent endpoints.

Pola Aman: Pisahkan I/O Eksternal

Selesaikan seluruh panggilan jaringan eksternal sebelum meminjam koneksi database.

REFAKTORING BERSIH

ProcessOrderSafe mengeksekusi externalCall() terlebih dahulu. Baru setelah I/O sukses, fungsi meminjam koneksi database secara singkat dan melepasnya via defer.

VERIFIKASI KONKURENSI

TestSafeProcessingConcurrently menguji 20 request konkuren pada pool berukuran 5. Seluruh 20 request sukses tuntas tanpa satupun mengalami timeout atau error.

Borrow connections at the last possible moment, release them immediately.

Uji 4: Deadlock via Pool-Locking

Kebuntuan fatal saat satu alur kerja menuntut banyak koneksi sekaligus.

MEKANISME DEADLOCK

Goroutine memegang conn1 lalu meminta conn2 pada pool berukuran 1. Karena pool kosong, permintaan conn2 memblokir hingga timeout tanpa pernah melepas conn1.

FORMULA PENCEGAHAN

pool_size = Tn * (Cm - 1) + 1. Di mana Tn = jumlah thread maksimum, Cm = koneksi maksimum per thread. TestPoolLockingDeadlock mengonfirmasi timeout deterministik.

Multi-connection workflows require explicit lower-bound pool sizing formulas.

Propagasi Error & Idempotensi

Koneksi harus dijamin kembali ke pool bahkan saat eksekusi query gagal.

ERROR PROPAGATION

TestExternalCallErrorPropagation memverifikasi saat externalCall menghasilkan error, galat diteruskan utuh ke pemanggil dan activeConns kembali tepat ke 0.

PRE-CANCELLED CONTEXT

TestPreCancelledContextProcessOrderSafe membuktikan jika context sudah dibatalkan sejak awal, query langsung ditolak tanpa sempat membakar slot koneksi pool.

Guaranteed defer cleanup prevents resource leaks even during abrupt error returns.

Matriks Pengujian Otomatis Lab

Sepuluh pengujian lulus sempurna tanpa data race (10/10 PASS).

CAKUPAN UJI

DirectOverhead (0.03s), OversizedPool (0.02s), StarvationDueToLeak (0.02s), SafeConcurrently (0.01s), PoolLockingDeadlock (0.05s), MockConnDoubleClose (0.00s).

HASIL VERIFIKASI

ErrorPropagation, PreCancelledContext, UnsafeLeakExecFailure, dan TotalCreatedPoolReuse lulus 100% di bawah go test -race tanpa race condition.

Every single failure mode is verified by an automated regression test assertion.

Solusi Skala Besar: Pooling Proxies

Multiplexing ribuan koneksi aplikasi menjadi puluhan koneksi database riil.

PERAN PGBOUNCER

PgBouncer dalam Transaction Mode mengembalikan koneksi server ke pool segera setelah transaksi commit/rollback selesai, memangkas kebutuhan koneksi hingga 90%.

AWS RDS PROXY

Mengelola pooling terpusat di tingkat infrastruktur cloud, menjaga kestabilan backend database saat aplikasi mengalami auto-scaling ratusan instance mendadak.

Connection proxies bridge high application concurrency with strict database limits.

Caveats Mode Proxy & Fitur Sesi

Transaction pooling melarang beberapa fitur status sesi pada PostgreSQL.

FITUR YANG RUSAK

Transaction mode memutus: SET/RESET sesi variabel, LISTEN/NOTIFY, WITH HOLD CURSOR, PREPARE/DEALLOCATE, temporary tables, dan advisory locks level sesi.

REKOMENDASI AUDIT

Sebelum beralih ke mode pooling transaksi PgBouncer, audit seluruh ORM dan query aplikasi untuk memastikan tidak ada ketergantungan pada status sesi lokal.

Transaction pooling optimizes scale but disables session-level database features.

Metrik Observabilitas: Pantau Pool

Jangan hanya memantau CPU server; amati metrik antrean internal pool.

METRIK KUNCI

1. Connection Wait Time (waktu antre meminjam soket). 2. In-use vs Idle Connections ratio. 3. Query timeout count. 4. Status pg_stat_activity 'idle in transaction'.

GEJALA TERSEMBUNYI

Kebocoran koneksi kerap memperlihatkan CPU dan RAM database yang sangat rendah dan tenang, sementara aplikasi di sisi pengguna mengalami timeout total.

Monitor connection wait times and pool saturation, not just server CPU usage.

Studi Kasus Oracle: 2048 ke 96

Memangkas 95% ukuran pool justru mendongkrak performa lebih dari 50x lipat.

EXPERIMENT FINDING

Oracle Real-World Performance group mendemonstrasikan penurunan koneksi dari 2048 ke 96 memangkas latensi kueri dari ~100ms menjadi ~2ms dengan throughput naik.

STATUS BUKTI

Status bukti: MEDIUM (demonstrasi vendor). Namun prinsip dasarnya sejalan dengan analisis PostgreSQL wiki: eliminasi kontensi CPU menghasilkan throughput bersih.

Fewer concurrent connections yield dramatically lower context switching and latency.

Anti-Patterns & Kesalahan Umum

Tujuh kebiasaan keliru yang memperparah krisis koneksi di lingkungan produksi.

KESALAHAN KONFIGURASI

1. Membesarkan pool saat koneksi error. 2. Menghitung pool per pod tanpa mengalikan replika. 3. Mengira koneksi idle gratis tanpa alokasi memori RAM.

KESALAHAN KODE

4. Memanggil HTTP API pihak ketiga di dalam blok transaksi aktif. 5. Tidak menyetel context timeout. 6. Mengabaikan kuota reserved superuser slots.

Never treat database connection pool expansion as a quick fix for latency problems.

Checklist Rilis Rekayasa Sistem

Daftar periksa verifikasi arsitektur sebelum menggelar layanan ke produksi.

CHECKLIST KONEKSI

[ ] Total (pods x pool_size) < max_connections server dikurangi slot reserved.
[ ] Bebas pemanggilan I/O eksternal di dalam blok pemegang koneksi.
[ ] Defer conn.Close() terpasang konsisten di seluruh branching error.

CHECKLIST MONITORING

[ ] Context timeout disematkan pada seluruh akuisisi & kueri.
[ ] Alerting aktif pada Connection Wait Time dan pool saturation.
[ ] Ukuran pool divalidasi via load testing empiris berbasis beban nyata.

Architectural discipline and strict connection hygiene guarantee production stability.

SOFTWARE ENGINEERING LAB | LAB 19

Keep Pools Lean, Multiplex at Scale, Prevent Leaks.

Database connection capacity is bounded by hardware resources, not by application thread concurrency. Never hold database connections during external network I/O.

- Verifikasi Otomatis: 10/10 PASS (go test -race ./tests/...)
- Kunci Arsitektur: Isolasi I/O eksternal, formula ((cores*2)+spindles), PgBouncer proxying
- Repositori: github.com/lukman-ss/software-engineering-lab/tree/main/labs/19-database-connection-pooling
- Komitmen Terverifikasi: ed2fe192cf5794f4430348df73e174611246be0e

Karya: Lukman / lukman-ss • Software Engineering Lab Seri 19